

RESEARCH ARTICLE

Twitter Sentiment Mining Suite: A TF-IDF and Logistic Regression Framework for Social Media Opinion Classification

Group Members

Ahmad Mustafa 2023-BS-AI-016

Muneeb Sajid 2023-BS-AI-028

Javed Mehmood 2023-BS-AI -029

Masood Bakhtiar 2023-BS-AI -056

Department of Computer Science, The University of Faisalabad, Faisalabad, 38000, Punjab, Pakistan

Abstract

Microblogging platforms generate large volumes of short, informal, and rapidly changing text, making automated sentiment analysis an important task for social media analytics. This paper presents the Twitter Sentiment Mining Suite, a machine learning framework for binary sentiment classification using natural language preprocessing, TF-IDF vectorization, and Logistic Regression. The implementation uses two extracted datasets: a training corpus of 1,600,000 tweets with six attributes and a testing corpus of 3,142,404 records with three attributes. The preprocessing pipeline normalizes case, removes hyperlinks, mentions, hashtags, non-alphabetic symbols, extra spaces, and English stop words. A baseline Logistic Regression model trained on 5,000 unigram TF-IDF features produced a testing accuracy of 36.87% on the available evaluation file; however, label-space inconsistency in the testing metadata makes this baseline result less reliable as a binary generalization estimate. An optimized Logistic Regression configuration using 10,000 unigram and bigram TF-IDF features with sublinear term-frequency scaling achieved 78.85% training convergence accuracy. The resulting framework demonstrates the workflow of scalable text mining, sentiment feature engineering, and interpretable linear classification for social media opinion analysis.

Key words: Sentiment Analysis, Twitter Mining, NLP, TF-IDF, Logistic Regression, Text Classification, NLTK, Scikit-learn, WordCloud

methodology, including dataset characteristics, preprocessing, feature extraction, model training, and evaluation metrics. Section 5 reports the experimental outcomes obtained from the notebook, including dataset profile, baseline evaluation, optimized training performance, confusion matrix interpretation, word-cloud visualization, and live inference behavior. Section 6 discusses the findings, limitations, and practical implications. Section 7 outlines the contributions of the work, and Section 8 concludes the paper with future research directions.

3. Literature Review

3.1. Sentiment Analysis in Social Media

Sentiment analysis is a natural language processing task that aims to detect the opinion polarity expressed in text. Social media sentiment analysis is more challenging than traditional document classification because tweets often contain incomplete grammar, creative spelling, domain-specific slang, short context windows, and implicit emotional cues. Early studies on Twitter sentiment demonstrated that distant supervision and emoticon-labeled data could be used to train scalable classifiers for short text. This project follows the same general research direction by treating tweets as short documents whose polarity can be inferred from lexical patterns.

3.2. TF-IDF for Text Representation

Machine learning models require numerical representations of text. Term Frequency-Inverse Document Frequency (TF-IDF) is a classical weighting scheme that assigns higher weights to terms that are frequent in one document but less common across the whole corpus. It is widely used for text classification because it converts raw language into sparse feature vectors while reducing the influence of very common terms. In this project, TF-IDF is used in two configurations: a baseline unigram model with 5,000 features and an optimized unigram-bigram model with 10,000 features and sublinear term-frequency scaling.

Introduction

Online social networks have become major sources of public opinion, product feedback, political reaction, and personal expression. Among these platforms, Twitter-style microblogging is especially difficult for computational analysis because posts are short, informal, noisy, and frequently contain links, user mentions, hashtags, emojis, abbreviations, sarcasm, and mixed emotional statements. Manual interpretation of millions of tweets is not practical; therefore, automated sentiment classification is required to transform raw user-generated text into usable opinion signals.

This study introduces a Python-based sentiment mining framework that classifies tweets into positive and negative sentiment classes. The project follows a complete machine learning workflow: dataset extraction from Google Drive, corpus loading, linguistic cleaning, sparse numerical feature generation, baseline classification, visualization of positive vocabulary, model re-engineering, and live sentence-level inference. The central objective is not only to build a classifier, but also to demonstrate how natural language preprocessing and vector-space modeling influence the quality of sentiment prediction.

The implemented approach uses TF-IDF features and Logistic Regression. This design is appropriate for an academic text mining project because it remains interpretable, computationally efficient, and suitable for large-scale sparse text matrices. Unlike deep learning models, the selected approach can be trained with comparatively lower hardware requirements while still providing a strong baseline for binary sentiment classification.

2. Paper Structure

The remainder of this paper is organized as follows. Section 3 reviews relevant work in social media sentiment analysis, TF-IDF representation, and linear text classifiers. Section 4 explains the

3.3. Logistic Regression for Text Classification

Logistic Regression is commonly used for binary classification on high-dimensional sparse features. In text classification, each vocabulary term or phrase becomes a feature, and the model estimates how strongly the feature contributes to positive or negative sentiment. Its advantages include fast training, probabilistic interpretation, and strong performance on bag-of-words and TF-IDF representations. This makes it a suitable baseline model for a large Twitter sentiment corpus.

3.4. Research Gaps and Contribution

Many beginner-level sentiment projects stop after training a model and reporting a single score. The present project extends the basic workflow by adding dataset profiling, text cleaning, baseline evaluation, optimization through feature-space expansion, visual exploration through word clouds, and sentence-level stress testing. The notebook also exposes an important evaluation issue: when test labels do not follow the same binary structure as training labels, reported accuracy can be misleading. This observation is valuable because data alignment is a frequent practical problem in applied machine learning.

4. Methodology

4.1. Dataset Description

The project uses two CSV files extracted from compressed archives stored in Google Drive. The training file is named `training.1600000.processed.noemoticon.csv` and contains 1,600,000 records with six raw columns. The second file, `twitter_dataset.csv`, contains 3,142,404 records with three columns. The training corpus is balanced, with 800,000 negative samples labeled 0 and 800,000 positive samples labeled 4 before binary remapping.

Table 1. Training Dataset Structure

Column	Assigned Name	Description
0	target	Sentiment class; 0 negative and 4 positive before remapping
1	id	Tweet identifier
2	date	Timestamp of tweet creation
3	flag	Query flag field
4	user	Twitter user name
5	tweet	Raw textual tweet content

Table 2. Dataset Profile Observed in Notebook

Property	Observed Value
Training corpus shape	1,600,000 rows x 6 columns
Testing corpus shape	3,142,404 rows x 3 columns
Missing values in training data	0 missing values in all six columns
Negative training labels	800,000 records labeled 0
Positive training labels	800,000 records labeled 4
Binary remapping for optimized model	0 -> 0 and 4 -> 1

4.2. Tools and Libraries

The implementation is written in Python and executed in Google Colab. Google Drive is mounted for dataset extraction, while standard data science libraries are used for text processing, visualization, vectorization, and model training. Table 3 summarizes the major libraries used in the notebook.

Table 3. Libraries and Their Roles

Library	Role in Project
os, zipfile	Drive path handling and extraction of ZIP datasets
re	Regular-expression based tweet cleaning
NumPy, Pandas	Numerical and tabular data processing
NLTK	English stop-word loading for preprocessing
Scikit-learn	TF-IDF vectorization, Logistic Regression, and metrics
Seaborn, Matplotlib	Confusion matrix visualization
WordCloud	High-frequency positive sentiment term visualization

4.3. Linguistic Preprocessing Pipeline

The preprocessing function `clean_tweet` transforms each raw tweet into a standardized representation. The function first converts all text to lowercase. It then removes hyperlinks, user mentions, hashtags, numbers, punctuation, and non-alphabetic symbols. Multiple spaces are compressed into a single space, and English stop words are removed using NLTK. This sequence reduces noise and produces a compact token stream suitable for TF-IDF extraction.

The preprocessing workflow can be represented as: raw tweet -> lowercase normalization -> URL removal -> mention removal -> hashtag removal -> alphabetic filtering -> whitespace normalization -> stop-word removal -> cleaned tweet.

Table 4. Text Cleaning Operations

Operation	Purpose
Lowercase conversion	Reduces duplicate vocabulary caused by case variation
URL removal	Eliminates external links that do not directly express sentiment
Mention removal	Removes user references before feature extraction
Hashtag removal	Drops standalone tracking tokens from the raw text
Non-alphabetic filtering	Removes punctuation, digits, emojis, and symbols
Whitespace compression	Normalizes spacing after regular-expression filtering
Stop-word removal	Reduces very common low-information words

4.4. Feature Extraction with TF-IDF

Two vectorization configurations are implemented. The baseline vectorizer uses 5,000 unigram features. The optimized vectorizer increases the maximum feature space to 10,000 and includes both unigrams and bigrams. It also applies sublinear term-frequency scaling, which reduces the dominance of repeated words by replacing raw frequency with a logarithmic form.

The general TF-IDF weighting scheme is expressed as:

$$\text{TF-IDF}(t,d) = \text{TF}(t,d) \times \log(N / \text{DF}(t))$$

where $\text{TF}(t,d)$ is the frequency of term t in document d , N is the total number of documents, and $\text{DF}(t)$ is the number of documents containing term t .

Table 5. Vectorization Configurations

Configuration	Vocabulary Limit	N-Gram Range	Sublinear TF
Baseline TF-IDF	5,000	Unigrams only	No
Optimized TF-IDF	10,000	Unigrams and bigrams (1,2)	Yes

4.5. Classification Algorithm

The classifier used in both stages is Logistic Regression. The baseline model is initialized with `max_iter = 1000` to support convergence on the large sparse feature matrix. The optimized model keeps `max_iter = 1000` and sets `C = 5`, which lowers the regularization strength and allows the model to fit richer decision boundaries from the expanded TF-IDF feature set.

For a feature vector x , Logistic Regression estimates the probability of a positive sentiment class as:

$$P(y = 1 | x) = 1 / (1 + \exp(-(\beta_0 + \beta^T x)))$$

4.6. System Architecture

The architecture consists of seven functional layers: dataset storage in Google Drive, ZIP extraction into the Colab workspace, Pandas-based corpus loading, regex and NLTK-based text preprocessing, TF-IDF vectorization, Logistic Regression model training, and evaluation or live inference. The live inference function applies the same cleaning and vectorization steps to unseen text before returning a positive or negative label.

Table 6. System Modules

Module	Function
Dataset Access Layer	Mounts Google Drive and extracts train/test ZIP files
Corpus Loading Layer	Loads CSV files using ISO-8859-1 encoding
Preprocessing Layer	Cleans tweets and removes noise tokens
Feature Engineering Layer	Converts cleaned tweets into TF-IDF sparse vectors
Modeling Layer	Trains Logistic Regression classifiers
Evaluation Layer	Computes accuracy, classification report, and confusion matrix
Inference Layer	Predicts sentiment for manually entered sentences

4.7. Evaluation Metrics

The notebook evaluates classification behavior through accuracy, precision, recall, F1-score, and confusion matrix analysis. Accuracy measures the overall proportion of correct predictions. Precision measures the reliability of positive predictions for each class. Recall measures how many actual examples of a class are recovered. F1-score combines precision and recall. The confusion matrix visualizes correct and incorrect predictions across label categories.

5. Results

5.1. Dataset Exploration Results

The training corpus was loaded successfully with 1.6 million tweet records and no missing values. The original training class distribution is balanced, with equal negative and positive classes. A sample cleaning operation transformed the raw tweet "@switchfoot <http://twitpic.com/2y1zl> - Awww, that's a bummer. You shoulda got David Carr of Third Day to do it. ;D" into "thats bummer shoulda got david carr third day". This confirms that hyperlinks, mentions, punctuation, and stop words were removed as intended.

5.2. Baseline Model Performance

The baseline model used 5,000 unigram TF-IDF features and a Logistic Regression classifier. The observed testing accuracy was

0.368729, or 36.87%. The classification report showed strong performance only for class 0, while classes 1 and 2 had zero precision and recall. The model also generated predictions for class 4 even though the filtered evaluation report contained no support for that class. This indicates a mismatch between the training label space and the testing label metadata, so the baseline accuracy should be interpreted cautiously rather than treated as a clean binary sentiment score.

Table 7. Baseline Classification Report

Class	Precision	Recall	F1-Score	Support
0	0.79	0.74	0.76	1,570,067
1	0.00	0.00	0.00	1,561,529
2	0.00	0.00	0.00	10,725
4	0.00	0.00	0.00	0
Accuracy	-	-	0.37	3,142,321
Macro Avg.	0.20	0.18	0.19	3,142,321
Weighted Avg.	0.39	0.37	0.38	3,142,321

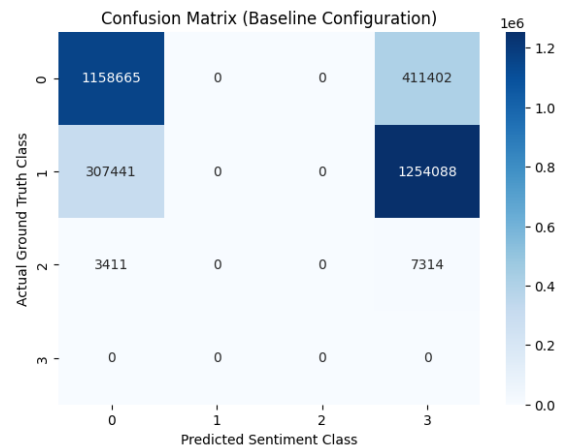


Fig. 1. Confusion matrix generated for the baseline Logistic Regression configuration.

5.3. Optimized Model Training Result

After remapping the training labels from $\{0,4\}$ to $\{0,1\}$, the optimized model used 10,000 TF-IDF features with unigram and bigram coverage and sublinear term-frequency weighting. This configuration achieved a training convergence accuracy of 78.85%. Because the notebook does not report an independent optimized test-set accuracy after binary label alignment, this value should be reported as training accuracy rather than final deployment accuracy.

Table 8. Model Configuration Comparison

Model Stage	Feature Setting	Classifier Setting	Observed Result
Baseline	5,000 unigram TF-IDF features	Logistic Regression	Test acc.: 36.87%; label mismatch
Optimized	10k unigram + bigram; sublinear TF	Logistic Regression, C=5	Train acc.: 78.85%

5.4. Word-Cloud Visualization

The positive sentiment vocabulary was visualized using a WordCloud generated from the first 20,000 cleaned positive tweets. Terms such as love, thank, good, today, lol, new, and work appear prominently, indicating that the positive subset contains frequent expressions of appreciation, pleasant events, social response, and everyday activity. This visualization is exploratory and does not replace quantitative evaluation, but it helps interpret the lexical content learned by the model.



5.5. Live Sentiment Inference Stress Testing

Table 9. Selected Live Inference Outputs

5.6. Error Pattern Observations

6. Discussion

© The Author 2026. Prepared for academic submission. All rights reserved.

positive vocabulary, optimizes the feature space, and performs live sentiment inference on complex example sentences.

The baseline configuration produced a testing accuracy of 36.87%, but the classification report and warnings indicate label-space inconsistency in the evaluation file. The optimized configuration, based on 10,000 unigram-bigram TF-IDF features and Logistic Regression with $C = 5$, achieved 78.85% training accuracy. Stress tests showed that the model can detect many direct emotional cues but still struggles with sarcasm, negation, mixed sentiment, and slang. The main value of the work is its complete and interpretable NLP workflow, while future development should focus on clean validation, richer preprocessing, and comparison with advanced contextual language models.

References

1. Pang, B., Lee, L., & Vaithyanathan, S. Thumbs up? Sentiment classification using machine learning techniques. Proceedings of EMNLP, pages 79-86, 2002.
2. Go, A., Bhayani, R., & Huang, L. Twitter sentiment classification using distant supervision. CS224N Project Report, Stanford University, 2009.
3. Liu, B. Sentiment Analysis and Opinion Mining. Morgan & Claypool Publishers, 2012.
4. Manning, C. D., Raghavan, P., & Schütze, H. Introduction to Information Retrieval. Cambridge University Press, 2008.
5. Ramos, J. Using TF-IDF to determine word relevance in document queries. Proceedings of the First Instructional Conference on Machine Learning, 2003.
6. Hosmer, D. W., Lemeshow, S., & Sturdivant, R. X. Applied Logistic Regression. Wiley, 3rd edition, 2013.
7. Bird, S., Klein, E., & Loper, E. Natural Language Processing with Python. O'Reilly Media, 2009.
8. Pedregosa, F., et al. Scikit-learn: Machine learning in Python. Journal of Machine Learning Research, 12:2825-2830, 2011.
9. Hunter, J. D. Matplotlib: A 2D graphics environment. Computing in Science & Engineering, 9(3):90-95, 2007.
10. Jurafsky, D., & Martin, J. H. Speech and Language Processing. Draft 3rd edition, 2023.